Zero to One Diagnostic — Outlier

Zero to One Diagnostic

Roshan Abraham — Outlier · github.com/rosh100yx/outlier · npm: `outlier-audit`

You don't have the problem you think you have. And you've probably been working very hard to avoid finding out.

A diagnostic to test whether you're solving the right problem before you commit more resources to the wrong one.

Problem–Hypothesis Canvas

Block	Your answer
The Problem	"My team uses AI agents. I have no idea who's delegating what, how much of the shipped code is actually theirs, or whether the agent is creating more work than it saves. The only signal I get is a cloud token bill. I'm flying blind on adoption."
Who Feels It	Engineering leads at AI-native startups and scaleups with 5–30 engineers who recently switched to Claude Code, Cursor, or Aider. Specifically: leads who do sprint retros and need to know whether the team is learning or just prompting, architecting or just reviewing, building or just refactoring agent output.
Evidence	• METR July 2025 RCT: experienced devs with AI 19% slower, believed 20% faster — adoption without measurement creates invisible tax. • 3 engineering leads in the last 30 days say "we use AI" but cannot answer: "what % of HEAD is machine-written?" • No CLI or dashboard today shows per-dev surviving-line authorship, agent reuse

Block	Your answer
	vs. duplication, or blast radius at the delegation point.
Founder Fit	While building with an AI agent, I couldn't tell whether I was architecting or just reviewing. The token bill was there. The authorship receipt was not.
The Hypothesis	We believe engineering teams struggling with AI agent adoption lack a local instrument that shows execution share, structural reuse, and blast radius (agent controls and directives) at the delegation point. If we ship a CLI that reads git + agent logs for surviving-line authorship and flags redundancy, then leads will stop guessing about AI usage and start governing it.
Test in 30 Days	• v0.23.1 published on npm (done). • Run `outlier audit` on 3 repos by people besides me. • Get 1 engineering lead to say "I'd show this in my next retro." • 10 GitHub stars from non-me users. • If 0 external receipts → kill.
Kill Criteria	• Teams run it once out of curiosity, never again in a workflow. • The authorship signal collapses on repos >5K lines or with non-trivial merge patterns. • Only researchers care; no shipping team adopts it. • I stop caring after 3 months.

30-Day Validation Protocol

Pick ONE hypothesis: Engineering leads will use a local CLI receipt to govern AI-agent adoption if it shows per-dev authorship, reuse, and blast radius.

Go / No-Go Signal

GO if: ≥3 external audits + ≥10 stars + 1 lead says "I'd show this in a team retro."

NO-GO if: 0 external audits + <10 stars + no repeat-usage signal.

Smallest Real Test

Not a survey. Publish. Write one README receipt showing authorship + reuse. Post to one dev community. Watch whether someone runs it without instruction and says "I didn't know we were this far gone."

Build vs. Kill Filter

- [x] **Is the problem specific enough to describe in one sentence?**
  "I don't know how my team is actually adopting AI agents, and the only signal I have is a cloud token bill." → **YES**
- [x] **Have you spoken to 3 people who have this problem in the last 30 days?**
  → **YES** (3 leads, direct conversations)
- [ ] **Would someone pay to solve this before you built anything?**
  → Uncertain (OSS first, CI lane later). → **NOT YET**
- [x] **Do you care enough about this problem to still be working on it in 3 years?**
  → **YES**
- [x] **Can you describe what "wrong" looks like clearly enough to stop?**
  → **YES** (Kill Criteria above)

Score: 4–5 yes → Build. The 3-person validation is confirmed. The payment question is the remaining open gap.

Zero to One Diagnostic · Roshan Abraham · linkedin.com/in/rosh100yx · github.com/rosh100yx/outlier